

REDUX

PARTE C — REDUX TOOLKIT

Qui arriva il cuore della lezione.

Che cos'è Redux

Definizione. Redux è **un framework per la gestione e il mantenimento dello stato** di un'applicazione. Non è legato a React: funziona con qualsiasi framework frontend. `react-redux` è la libreria che fa da ponte fra i due.

Il punto di partenza, ripreso dalla trascrizione: il comportamento che hai costruito nel Progetto 9 con `useContext` + `useReducer` **simula** quello di Redux. La logica è identica: stato centrale, azioni, `dispatch`, `reducer`. Redux offre però molto di più.

I tre principi di Redux:

1. **Single source of truth** — un unico oggetto contiene tutto lo stato dell'applicazione.
2. **Lo stato è in sola lettura** — si cambia solo `dispatchando` azioni.
3. **I cambiamenti avvengono con funzioni pure** — i `reducer`.

Cosa aggiunge rispetto a `Context` + `Reducer`:

	Context + useReducer	Redux Toolkit
Strumenti di debug	nessuno	Redux DevTools: cronologia delle azioni, <code>time-travel</code>
Gestione asincrona	a mano	<code>createAsyncThunk</code> , <code>middleware</code>
Render inutili	tutti i consumatori si ridisegnano	<code>useSelector</code> ridisegna solo chi usa quel valore
Struttura su progetti grandi	si sfilaccia	slice separate per dominio
Boilerplate	poco	poco (con Toolkit)

La regola pratica: per un'app piccola o media, `Context` + `useReducer` basta. Redux ha senso quando lo stato è ampio, condiviso e cambia spesso — e soprattutto quando serve poter *ispezionare* cosa sta succedendo.

Standard Configuration — lo Store

```
npm install @reduxjs/toolkit react-redux
```

Lo store è l'oggetto unico che contiene tutto lo stato.

```
// src/redux/store.js
import { configureStore } from '@reduxjs/toolkit';
```

```
import apiReducer from './reducers/apiReducer';
import cartReducer from './reducers/cartReducer';
```

```
const store = configureStore({
  reducer: {
    api: apiReducer,
    cart: cartReducer
  }
});
```

```
export default store;
```

`configureStore` fa gratis tre cose che nel Redux classico andavano configurate a mano: combina i reducer, collega i **Redux DevTools**, e installa i middleware standard (fra cui i controlli che avvertono se modifichi lo stato per sbaglio).

Il collegamento all'app:

```
// index.js
import { Provider } from 'react-redux';
import store from './redux/store';
```

```
<Provider store={store}>
  <ThemeProvider theme={theme}>
    <App />
  </ThemeProvider>
</Provider>
```

È esattamente lo stesso pattern del `ThemeProvider` e dell'`AppProvider` visti nella Lezione 4: un componente che avvolge l'albero e rende un valore disponibile ovunque.

createSlice e Immer

Definizione. Una *slice* è una fetta dello stato con i suoi reducer e le sue azioni, tutto in un file.

```
import { createSlice } from '@reduxjs/toolkit';
```

```
const exampleSlice = createSlice({
```

```

name: 'example',
initialState: { value: 0 },
reducers: {
  increment: (state) => {
    state.value += 1;    // sembra una mutazione!
  },
  decrement: (state) => {
    state.value -= 1;
  }
}
});

```

```

export const { increment, decrement } = exampleSlice.actions;
export default exampleSlice.reducer;

```

Il punto che sorprende tutti, e che la trascrizione spiega bene. Nel Progetto 9 eri obbligato a scrivere:

```
return { ...state, value: state.value + 1 };
```

Qui scrivi `state.value += 1` e funziona. **Non stai violando l'immutabilità.**

Perché. Redux Toolkit include **Immer**, una libreria che lavora dietro le quinte. Immer ti passa una *bozza* (draft) dello stato: tu la modifichi come se fosse un oggetto normale, e Immer produce un nuovo oggetto immutabile, applicando solo le differenze. Ottieni la leggibilità della mutazione con la sicurezza dell'immutabilità.

L'unica regola da ricordare: o modifichi la bozza **oppure** ritorni un nuovo stato. Mai entrambe le cose nella stessa funzione.

// OK

```
increment: (state) => { state.value += 1; }
```

// OK

```
reset: () => ({ value: 0 })
```

// SBAGLIATO

```
strano: (state) => { state.value += 1; return { value: 0 }; }
```

La generazione automatica delle azioni. `createSlice` crea da solo le action creator a partire dai nomi dei reducer. `increment()` produce `{ type: 'example/increment' }`. Il formato `nomeSlice/nomeReducer` conta: ci torneremo con `addMatcher`.

Confronto con il Progetto 9: lì dovevi definire a mano le costanti delle action type, importarle nel reducer e nel componente, e scrivere le funzioni di dispatch. Qui è tutto generato.

useSelector e useDispatch

Definizione. I due hook che collegano un componente a Redux.

- **useSelector** legge un valore dallo store.
- **useDispatch** ritorna la funzione per spedire le azioni.

```
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement } from '../redux/reducers/exampleReducer';

const Contatore = () => {
  const value = useSelector((store) => store.example.value);
  const dispatch = useDispatch();

  return (
    <div>
      <h2>{value}</h2>
      <button onClick={() => dispatch(increment())}>+</button>
      <button onClick={() => dispatch(decrement())}>-</button>
    </div>
  );
};
```

Come funziona useSelector. Riceve l'intero store e tu ritorni la parte che ti interessa. Se scrivi `useSelector(store => store)` ottieni tutto, e in console vedi l'oggetto completo con tutte le slice — è il modo suggerito nella trascrizione per esplorare la struttura.

La regola d'oro delle performance: seleziona il valore più specifico possibile.

// Male: il componente si ridisegna a ogni cambiamento di QUALSIASI cosa nel carrello

```
const cart = useSelector((store) => store.cart);
```

// Bene: si ridisegna solo quando cambia il totale

```
const total = useSelector((store) => store.cart.total);
```

Questo è il vantaggio concreto rispetto al Context: con Context, ogni cambiamento del valore del provider ridisegna **tutti** i consumatori. Con `useSelector`, React-Redux confronta il valore selezionato e ridisegna solo chi è realmente interessato.

Attenzione a non creare oggetti nel selettore:

```
// Male: nuovo oggetto ad ogni chiamata → sempre "diverso" → render infiniti
const { total, items } = useSelector((s) => ({ total: s.cart.total, items: s.cart.items }));

// Bene: due selettori separati
const total = useSelector((s) => s.cart.total);
const items = useSelector((s) => s.cart.items);
```

Api Reducer — la slice dei dati

```
// redux/reducers/apiReducer.js
import { createSlice } from '@reduxjs/toolkit';
import client from '../api';

const initialState = {
  loading: true,
  error: { status: false, message: "" },
  photos: [],
  page: 1,
  totalPages: 0,
  query: ""
};

const apiSlice = createSlice({
  name: 'api',
  initialState,
  reducers: {
    startLoading: (state) => {
      state.loading = true;
      state.photos = [];
    }
  }
});
```

```

    },
    stopLoading: (state) => {
      state.loading = false;
    },
    saveData: (state, action) => {
      state.photos = action.payload;
      state.error = { status: false, message: " " };
    },
    catchError: (state, action) => {
      state.error.status = true;
      state.error.message = action.payload;
      state.photos = [];
    }
  }
});

```

```

export const { startLoading, stopLoading, saveData, catchError } = apiSlice.actions;
export default apiSlice.reducer;

```

Nota sulla sintassi dell'oggetto, che la trascrizione sottolinea: `initialState`, da solo funziona perché in JavaScript, quando la chiave ha lo stesso nome della variabile, si può omettere `initialState: initialState`. Si chiama *shorthand property*.

Api Response — la chiamata asincrona

Il corso adotta l'approccio **esplicito**: una funzione asincrona che dispatcha le azioni a mano.

```

// nel componente o in un helper
const fetchPhotos = async (dispatch, page) => {
  dispatch(startLoading());
  try {
    const { data } = await client.get('/photos', {
      params: { page, per_page: 12 }
    });
    dispatch(saveData(data));
  }
};

```

```

} catch (error) {
  if (error.response?.status === 403) {
    dispatch(catchError("Limite orario raggiunto"));
  } else {
    dispatch(catchError("Errore nel caricamento"));
  }
} finally {
  dispatch(stopLoading());
}
};

```

L'alternativa moderna: createAsyncThunk. Redux Toolkit offre uno strumento che automatizza questo schema, generando tre azioni (pending, fulfilled, rejected):

```

import { createAsyncThunk } from '@reduxjs/toolkit';

export const fetchPhotos = createAsyncThunk(
  'api/fetchPhotos',
  async (page, thunkAPI) => {
    try {
      const { data } = await client.get('/photos', { params: { page, per_page: 12 } });
      return data;
    } catch (error) {
      return thunkAPI.rejectWithValue(error.response?.status === 403
        ? "Limite orario raggiunto"
        : "Errore nel caricamento");
    }
  }
);

```

```

extraReducers: (builder) => {
  builder
    .addCase(fetchPhotos.pending, (state) => { state.loading = true; })
    .addCase(fetchPhotos.fulfilled, (state, action) => {

```

```

    state.loading = false;

    state.photos = action.payload;
  })

  .addCase(fetchPhotos.rejected, (state, action) => {

    state.loading = false;

    state.error = { status: true, message: action.payload };

  });
}

```

L'approccio del corso è più verboso ma **didatticamente migliore**: vedi esattamente cosa succede a ogni passo. Una volta capito, `createAsyncThunk` è la scelta da preferire nei progetti reali.

Handle Loading & Error

```

const Home = () => {

  const { loading, error, photos } = useSelector((store) => store.api);

  if (loading) return <SkeletonGrid count={12} />;
  if (error.status) return <ErrorMessage message={error.message} />;
  if (photos.length === 0) return <EmptyState />;

  return <PhotoGrid photos={photos} />;
};

```

Quattro stati distinti, ciascuno con la sua interfaccia. È il pattern della Lezione 3, ma con i dati che arrivano da Redux invece che da `useState`.

Helpers — Rowalizer

Definizione. Una funzione di utilità che **divide un array in righe** di N elementi ciascuna, per comporre la griglia di foto.

Il perché, dalla trascrizione: l'autore vuole mostrare le foto incolonnate in sezioni da tre. Serve quindi un array di righe, ognuna con tre foto.

```

// utils/helpers.js

export const rowalizer = (array, itemPerRow = 3) => {

  const rows = Math.ceil(array.length / itemPerRow);

```

```

return Array.from({ length: rows }, (_, i) =>
  array.slice(i * itemPerRow, i * itemPerRow + itemPerRow)
);
};

```

```

rowalizer([1,2,3,4,5,6,7], 3);
// → [[1,2,3], [4,5,6], [7]]

```

Scomposto:

- `Math.ceil(7 / 3) = 3` → arrotondamento per eccesso, perché l'ultima riga può essere incompleta.
- `Array.from({ length: 3 }, ...)` → crea le tre righe.
- `slice(i*3, i*3+3)` → estrae il pezzo di array corrispondente.

```

{rowalizer(photos).map((row, i) => (
  <PhotoSection key={i} row={row} />
))}

```

La nota di metodo più importante di questa lezione: l'autore osserva che questa funzione *"ripete il pattern per creare la paginazione"*. È lo stesso algoritmo — dividere un array in blocchi — usato per due scopi diversi: righe visive e pagine. Riconoscere che due problemi apparentemente distinti hanno la stessa soluzione è esattamente ciò che distingue chi programma da chi copia codice.

Photo Component & Advanced Loading

Definizione. Il caricamento progressivo delle immagini: prima lo skeleton, poi la foto quando è effettivamente scaricata.

```

const Photo = ({ urls, alt_description, id }) => {
  const [loaded, setLoaded] = useState(false);

  return (
    <Box position="relative">
      {!loaded && <Skeleton width="100%" height="300px" />}
      <img
        src={urls.regular}
        alt={alt_description}
        onLoad={() => setLoaded(true)}

```

```

    style={{
      opacity: loaded ? 1 : 0,
      transition: 'opacity 0.3s',
      position: loaded ? 'static' : 'absolute'
    }}
  />
</Box>
);
};

```

Il punto: `loading: false` in Redux significa "la *risposta API* è arrivata", non "le immagini sono visibili". Sono due momenti diversi, e senza questa distinzione l'utente vede riquadri bianchi al posto delle foto.

`onLoad` è l'evento nativo dell'elemento `<img>` che scatta quando il file è scaricato. Lo stato è **locale al componente**, non in Redux: ogni foto carica per conto suo, e metterlo nello store sarebbe sovraingegneria.

Item Per Page e Helpers con useCallback

```
const ITEMS_PER_PAGE = 12; // divisibile per 3 → righe sempre complete
```

E l'ottimizzazione della Lezione 3 applicata qui:

```

const handleSearch = useCallback((query) => {
  dispatch(saveQuery(query));
  dispatch(setPage(1));
}, [dispatch]);

```

`dispatch` è garantito stabile da React-Redux, quindi la funzione mantiene lo stesso riferimento e i componenti figli avvolti in `memo` non si ridisegnano inutilmente.

Search Photos e Save Query

```

// nella slice
setQuery: (state, action) => {
  state.query = action.payload;
  state.page = 1; // la ricerca riparte da pagina 1
}

```

```

const searchPhotos = async (dispatch, query, page) => {
  dispatch(startLoading());
  try {
    const { data } = await client.get('/search/photos', {
      params: { query, page, per_page: ITEMS_PER_PAGE }
    });
    dispatch(saveData(data.results)); // nota: .results, non data
    dispatch(setTotalPages(data.total_pages));
  } catch (error) {
    dispatch(catchError("Errore nella ricerca"));
  } finally {
    dispatch(stopLoading());
  }
};

```

Due dettagli reali:

1. L'endpoint `/search/photos` ritorna un oggetto con `results`, `total` e `total_pages`, non un array diretto. Le API non sono uniformi: leggere la documentazione non è opzionale.
2. **Salvare la query nello store** permette di mantenere il contesto fra le pagine: cambiando pagina, l'app sa ancora cosa stavi cercando. Se la query fosse in uno `useState` locale, si perderebbe alla navigazione.

Pagination

```

// nella slice
setPage: (state, action) => { state.page = action.payload; },
nextPage: (state) => {
  state.page = state.page < state.totalPages ? state.page + 1 : state.page;
},
prevPage: (state) => {
  state.page = state.page > 1 ? state.page - 1 : state.page;
}

```

```

const Pagination = () => {

```

```

const { page, totalPages } = useSelector((store) => store.api);
const dispatch = useDispatch();
const pagine = Array.from({ length: totalPages }, (_, i) => i + 1);
return (
  <Stack direction="row" space={2}>
    <button onClick={() => dispatch(prevPage())} disabled={page === 1}><</button>
    {pagine.map((n) => (
      <button
        key={n}
        onClick={() => dispatch(setPage(n))}
        className={n === page ? 'attivo' : ''}
      >{n}</button>
    ))}
    <button onClick={() => dispatch(nextPage())} disabled={page === totalPages}>>>/button>
  </Stack>
);
};

```

E il caricamento automatico al cambio pagina:

```

useEffect(() => {
  if (query) {
    searchPhotos(dispatch, query, page);
  } else {
    fetchPhotos(dispatch, page);
  }
}, [page, query, dispatch]);

```

Il pattern architetturale: i bottoni non chiamano l'API. Cambiano solo lo stato **page**. Un `useEffect` osserva quel cambiamento e reagisce. Questo separa nettamente **cosa vuole l'utente** da **come si ottengono i dati**, ed è la struttura che rende manutenibile un'app di medie dimensioni.

Nota pratica su totalPages: Unsplash può ritornare migliaia di pagine. Elencarle tutte è impraticabile — in produzione si mostra una finestra di 5-7 pagine attorno a quella corrente.

=====

PARTE D — IL CARRELLO E I PATTERN AVANZATI

Cart Reducer

```
const cartSlice = createSlice({
  name: 'cart',
  initialState: { cart: [], total: 0, amount: 0 },
  reducers: {
    addToCart: (state, action) => {
      const esistente = state.cart.find((i) => i.id === action.payload.id);
      if (esistente) {
        esistente.quantity += 1;      // mutazione diretta, grazie a Immer
      } else {
        state.cart.push({ ...action.payload, quantity: 1 });
      }
    },
    clearCart: (state) => {
      state.cart = [];
      state.total = 0;
    }
  }
});
```

Confronta con il reducer del Progetto 9, dove servivano `map` con ternario e `spread` annidati. Qui `esistente.quantity += 1` e `push` fanno la stessa cosa, e Immer garantisce l'immutabilità. È la differenza concreta fra Redux classico e Toolkit.

ExtraReducers

Definizione. `extraReducers` permette a una slice di **rispondere ad azioni definite altrove**: in un'altra slice, o create a mano con `createAction`.

```
import { createSlice, createAction } from '@reduxjs/toolkit';

export const removeFromCart = createAction('cart/removeFromCart');
```

```
const cartSlice = createSlice({
  name: 'cart',
  initialState,
  reducers: { /* ... */ },
  extraReducers: (builder) => {
    builder.addCase(removeFromCart, (state, action) => {
      state.cart = state.cart.filter((item) => item.id !== action.payload.id);
    });
  }
});
```

A cosa serve davvero. Le azioni generate da `reducers` appartengono a quella slice sola. Con `extraReducers` una slice può **ascoltare** azioni esterne. Il caso tipico: `createAsyncThunk` genera azioni fuori dalla slice, e `extraReducers` è il modo per intercettarle. È anche il modo per far reagire due slice diverse alla stessa azione, senza dispatcharla due volte.

Il `builder` è un oggetto su cui si concatenano i casi da gestire — come il `switch` del Progetto 9, ma tipizzato e componibile.

AddMatcher

Definizione. `addMatcher` intercetta azioni in base a una **condizione**, non a un nome esatto. Riceve una funzione che ritorna `true` o `false`.

Il problema da risolvere, come lo pone la trascrizione: il totale del carrello va aggiornato sia quando si aggiunge sia quando si rimuove. Dispatchare un'azione separata per ricalcolarlo sarebbe ridondante.

```
const isCartAction = (action) => action.type.startsWith('cart/');

extraReducers: (builder) => {
  builder
    .addCase(removeFromCart, (state, action) => {
      state.cart = state.cart.filter((i) => i.id !== action.payload.id);
    })
    .addMatcher(isCartAction, (state) => {
      // gira DOPO ogni azione della slice cart
    })
}
```

```

const { total, amount } = state.cart.reduce(
  (acc, item) => {
    acc.total += item.price * item.quantity;
    acc.amount += item.quantity;
    return acc;
  },
  { total: 0, amount: 0 }
);
state.total = parseFloat(total.toFixed(2));
state.amount = amount;
});
}

```

Il meccanismo. Ricordi che `createSlice` genera i type nel formato `nomeSlice/nomeReducer`? Ecco a cosa serve. `action.type.startsWith('cart/')` cattura tutte le azioni del carrello con una riga.

Il risultato: un solo dispatch dell'utente aggiorna **due valori separati** — la lista e il totale. Non serve ricordarsi di chiamare `getTotals()` dopo ogni operazione, come nel Progetto 9.

L'ordine conta: i `addCase` vengono valutati per primi, poi i `addMatcher` nell'ordine di dichiarazione. Il matcher gira dopo che il case ha già modificato il carrello, quindi calcola sul dato aggiornato.

Multiple Matcher

Definizione. Più matcher nella stessa slice, ciascuno per una condizione diversa.

```

const isPending = (action) => action.type.endsWith('/pending');
const isRejected = (action) => action.type.endsWith('/rejected');
const isFulfilled = (action) => action.type.endsWith('/fulfilled');

extraReducers: (builder) => {
  builder
    .addMatcher(isPending, (state) => { state.loading = true; state.error.status = false; })
    .addMatcher(isFulfilled, (state) => { state.loading = false; })
    .addMatcher(isRejected, (state, action) => {
      state.loading = false;

```

```

    state.error = { status: true, message: action.payload };
  })
  .addDefaultCase((state) => state);
}

```

Il valore: con dieci thunk asincroni diversi, gestisci loading ed errore **una volta sola** invece di trenta `addCase`. È la forma più compatta possibile per una logica ripetitiva.

`addDefaultCase` è il `default` dello switch: gestisce tutto ciò che non ha trovato corrispondenza.

=====

PARTE E — I FORM (Formik e Yup)

Formik — gestire i form

Definizione. Formik è una libreria per la gestione dei form in React. Elimina la necessità di scrivere uno `useState` per ogni campo.

L'autore la definisce la sua libreria preferita, e nota un dettaglio importante: *"è fatto totalmente in React nativo"* — nessuna dipendenza esotica sotto il cofano.

```
npm install formik
```

Il problema. Nel Progetto 6 hai gestito un form con un oggetto di stato e un handler. Con cinque campi, validazione, messaggi di errore, gestione del `touched` (campo toccato ma lasciato vuoto) e stato di invio, il codice esplode.

```

import { Formik } from 'formik';

const Checkout = () => (
  <Formik
    initialValues={{ nome: "", cognome: "", indirizzo: "", civico: "", cap: "" }}
    onSubmit={(values) => {
      alert(JSON.stringify(values, null, 2));
    }}
  >
    {({ values, errors, touched, handleChange, handleBlur, handleSubmit, isSubmitting }) => (
      <form onSubmit={handleSubmit}>
        <Input name="nome"
          value={values.nome}

```

```

onChange={handleChange}
onBlur={handleBlur}
/>
{errors.nome && touched.nome && <Text variant="caption">{errors.nome}</Text>}
<Input
  name="cap"
  value={values.cap}
  onChange={handleChange}
  onBlur={handleBlur}
/>
{errors.cap && touched.cap && <Text variant="caption">{errors.cap}</Text>}
<button
  type="submit"
disabled={isSubmitting}
>Conferma ordine<
/button>
</form>
)}
</Formik>
);

```

La sintassi da decifrare — le render props. `<Formik>` non riceve JSX come children, ma **una funzione** che riceve lo stato del form e ritorna il JSX. È un pattern precedente agli hook per condividere logica, e Formik lo usa ancora.

Cosa fornisce:

<code>values</code>	i valori attuali di tutti i campi
<code>errors</code>	gli errori di validazione
<code>touched</code>	quali campi sono stati toccati e poi abbandonati
<code>handleChange</code>	un solo handler per tutti i campi (usa l'attributo <code>name</code>)
<code>handleBlur</code>	marca il campo come "toccato"
<code>handleSubmit</code>	fa già <code>preventDefault</code> e valida
<code>isSubmitting</code>	true durante l'invio, per disabilitare il bottone

Il ruolo di `touched`. Senza, l'errore "campo obbligatorio" apparirebbe già al primo caricamento, prima che l'utente abbia fatto qualsiasi cosa. La condizione `errors.nome && touched.nome` mostra l'errore solo dopo che l'utente è entrato nel campo e ne è uscito. È un dettaglio di UX che fa la differenza fra un form fastidioso e uno usabile.

Yup — validazione avanzata

Definizione. Yup è una libreria per costruire **schemi di validazione** dichiarativi. Si integra nativamente con Formik.

```
npm install yup
```

```
import * as Yup from 'yup';

const validationSchema = Yup.object({
  nome: Yup.string()
    .min(2, 'Troppo corto')
    .max(50, 'Troppo lungo')
    .required('Campo obbligatorio'),

  cognome: Yup.string()
    .required('Campo obbligatorio'),

  email: Yup.string()
    .email('Email non valida')
    .required('Campo obbligatorio'),

  cap: Yup.string()
    .matches(/^[0-9]{5}$/, 'Il CAP deve avere 5 cifre')
    .required('Campo obbligatorio'),

  civico: Yup.number()
    .positive('Deve essere positivo')
    .integer('Deve essere un numero intero')
    .required('Campo obbligatorio')
});
```

Si collega con una sola prop:

```

<Formik
  initialValues={{ ... }}
  validationSchema={validationSchema}
  onSubmit={(values, { setSubmitting, resetForm }) => {
    dispatch(clearCart());
    resetForm();
    setSubmitting(false);
  }}
>

```

Perché è un salto di qualità. La validazione non è più sparsa in una funzione con venti `if`: è uno **schema dichiarativo**, leggibile come un contratto. Formik lo esegue a ogni cambio e popola `errors` automaticamente.

Validazioni condizionali, per i casi reali:

```

partitaIva: Yup.string().when('tipoCliente', {
  is: 'azienda',
  then: (schema) => schema.required('Obbligatoria per le aziende'),
  otherwise: (schema) => schema.notRequired()
})

```

Attenzione: Yup valida sul client, quindi è **esperienza utente, non sicurezza**. Chiunque può disabilitare JavaScript e inviare dati arbitrari. La validazione server-side resta obbligatoria.

Nota sull'ecosistema attuale. Formik è ancora largamente usato, ma il suo sviluppo ha rallentato. Le alternative più attive oggi sono **React Hook Form** (più performante, basato su input non controllati) con **Zod** (alternativa a Yup, con tipizzazione TypeScript nativa). I concetti — schema di validazione, touched, errori — sono identici, quindi imparare Formik e Yup ti prepara comunque.